

AN1.3 - Agent Contract Closure (A1)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A1 - Contracts & Registry (declarative only; no runtime behavior) **Depends on:** AN1.1 audit, AN1.2 locked decisions (D1-D4, P1-P3, 8 invariants) **Gate:** G01 - must be clean before A2 (Tool Contract + Registry) begins.

1. Contract files

All under `frontend/types/agent-framework/` - the single declarative contract layer. Every file is pure: shapes, closed vocabularies, deterministic transition tables, pure validation predicates. No I/O, no DB, no network, no provider SDK, no scheduling.

File	Purpose	Key exports
<code>common.ts</code>	Shared contract primitives	<code>ContractViolation</code> , <code>ContractValidationResult</code> , <code>contractOk/contractResult</code> , <code>JsonSchema</code> , field predicates (<code>isUnitInterval</code> , <code>isIsoTimestamp</code> , <code>isIdentifier</code> , ...)
<code>autonomy-contract.ts</code>	Autonomy Levels 0-4	<code>AutonomyLevel</code> , <code>AUTONOMY_LABELS</code> , <code>DEFAULT_AUTONOMY_LEVEL = 0</code> , <code>MAX_CONFIGURABLE_AUTONOMY_V1 = 2</code> , <code>TRADING_DECISION_MAX_AUTONOMY_V1 = 1</code> , <code>LIVE_EXECUTION_MIN_AUTONOMY = 3</code> , <code>canRunAtAutonomy()</code>
<code>permission-contract.ts</code>	Default-deny permission model	<code>PermissionKey</code> (9 keys), <code>PermissionPolicy</code> (allowlist), <code>DANGEROUS_PERMISSIONS</code> , <code>LIVE_EXECUTION_DEFAULT = "DENY"</code> , <code>evaluatePermission()</code> , <code>validatePermissionPolicy()</code> (parser-layer half of the double gate)
<code>evidence-contract.ts</code>	Cited, immutable evidence	<code>AgentEvidenceType</code> (10), <code>AgentEvidence</code> , <code>EvidenceProvenance</code> , <code>EvidenceRef</code> , <code>validateAgentEvidence()</code>
<code>memory-contract.ts</code>	Provider-independent, policy-gated memory	<code>MemoryLayer</code> (7), <code>MemoryPolicy</code> , <code>MemoryRetention</code> , <code>AgentMemoryRecord</code> , <code>DEFAULT_MEMORY_POLICY</code> , <code>validateMemoryPolicy()</code> (<code>RUN_STATE</code> not table-backed)
<code>tool-contract.ts</code>	Declarative tool capability + Planner!=Executor boundary	<code>ToolCategory</code> (10), <code>ToolDefinition</code> (no handler), <code>ToolCreditCost</code> , <code>PlannerToolRequest /</code> <code>AuthorizedToolIntent /</code> <code>ExecutorInvocation</code> (the three roles), <code>ToolResult</code> , <code>validateToolDefinition()</code>

File	Purpose	Key exports
agent-run-contract.ts	DB-backed run state machine	AgentRunStatus (14), RUN_STATUS_TRANSITIONS (deterministic table), TERMINAL_RUN_STATUSES, AgentRun / AgentStep / AgentToolCall / AgentRunTrace, AgentRunLimits + DEFAULT_RUN_LIMITS + LIMIT_BREACH_STATUS, isValidRunTransition(), validateRunTransition()
agent-contract.ts	Versioned, vendor-neutral agent definition	AgentDefinition, AgentVersionSnapshot, ModelPolicy / ToolBinding / TriggerPolicy / CreditPolicy / KnowledgeSourceRef, validateAgentDefinition(), makeDefaultAgentDefinitionBase()
index.ts	Contract barrel + version	AGENT_FRAMEWORK_CONTRACT_VERSION = "AF-v1", re-exports all of the above

common.ts is a minor structural addition beyond the 8 files named in the A1 scope - a shared-primitives module so the leaf contracts don't cross-import or duplicate the violation/predicate vocabulary. It carries no domain shapes.

Contract is versioned: AGENT_FRAMEWORK_CONTRACT_VERSION = "AF-v1". Bumped only on a breaking change to an exported shape/vocabulary; additive changes (new optional field, appended enum member) do not bump it.

2. Registry

frontend/services/agent-framework/agent-type-registry.ts - the **code registry** that is the authority for agent types (AN1.2 decision D1: no Prisma enum).

- AgentType = the 8 locked canonical keys: RESEARCH, MARKET_INTELLIGENCE, STRATEGY_RESEARCH, BACKTEST_OPTIMIZATION, RISK, NEWS_EVENT, TRADING_DECISION, PORTFOLIO. The Supervisor/Orchestrator is deliberately **not** a type - it is runtime architecture (A5).
- AGENT_TYPE_REGISTRY: Record<AgentType, AgentTypeSpec> - frozen. Each spec: label, description, defaultTools[], defaultPermissions[] (never a dangerous permission), autonomyCap (**1 for every type in v1**), independentOfDecision (true for RISK).
- LEGACY_AGENT_TYPE_ALIASES - maps the 3 seeded rows + mock vocabulary (market-analyst -> MARKET_INTELLIGENCE, risk-manager -> RISK, strategy-generator -> STRATEGY_RESEARCH, trading-copilot -> TRADING_DECISION, news-researcher -> NEWS_EVENT, portfolio-advisor -> PORTFOLIO). DB rows are never rewritten. seo-writer / customer-support intentionally have no mapping - those agents stay on the frozen legacy layer.
- Helpers: isRegisteredAgentType(), isKnownAgentType() (canonical or alias - the predicate the definition validator uses), resolveAgentType(), getAgentTypeSpec(), autonomyCapForType().
- **Declarative:** imports only import type from @/types/agent-framework/*. Dependency direction is contracts <- registry <- runtime, never reversed.
- Adding EXECUTION / COMPLIANCE / PORTFOLIO_OPTIMIZER / STRATEGY_MONITOR / DATA_QUALITY later = one entry here, zero schema migration.

3. Type validation

`validateAgentDefinition(def, opts)` is the single entry point. Pure, no I/O. `opts.isRegisteredType / opts.autonomyCapForType` are **injected** (from the registry) so the contract file never imports `services/*`. Checks:

- required strings: `id`, `slug` (identifier form), `version`, `name`, `description`, `objective`, `instructions`
- `status` in `{draft, active, paused, archived}`
- `type` is non-empty **and** resolves against the registry (D1)
- `modelPolicy`: allowed non-empty; preferred in allowed; every fallback in allowed; `maxContextTokens` positive when set - **no vendor name anywhere**
- `creditPolicy`: all three ceilings positive; `perRunCeiling` $\leq$ `perDayCeiling`; `requireEstimateUnder` $\leq$ `perRunCeiling`
- `triggerPolicy`: valid shape; at least one trigger (manual / schedule / event)
- `tools`: no duplicate `toolId`; `creditCeiling` positive when set
- `knowledgeSources`: valid kind / id / readPolicy
- `permissionPolicy`: delegated to `validatePermissionPolicy()` - includes the **parser-layer half of the LIVE_EXECUTION = DENY double gate**
- `memoryPolicy`: delegated to `validateMemoryPolicy()`
- `autonomyLevel`: 0-4; $\leq$ `MAX_CONFIGURABLE_AUTONOMY_V1` (2); $\leq$ the registry cap for the type (so `TRADING_DECISION` above 1 fails)
- `outputSchema`: must be an object (JSON Schema)

Every failure is a `{ path, message }` pair - a field path and a plain-language reason, never a bare code (mirrors `IntelligenceAnalysisOutcome.evaluationBasis` discipline). An invalid result means the definition **cannot be saved** - there is no partial-save path.

4. Invalid-state tests

`frontend/scripts/validate-agent-contracts.ts` - 38 tests, house style (node:assert/strict, tsx), run via `npm run validate:agent-contracts`.

Deterministic-rejection coverage:

- **AgentDefinition**: unknown type; bad slug; empty objective / instructions; bad status; empty `modelPolicy.allowed`; fallback not in allowed; `perRunCeiling` $>$ `perDayCeiling`; no trigger; duplicate tool binding; null `outputSchema`; autonomy above v1 cap; autonomy above registry type cap; `CAN_EXECUTE_ORDER` grant rejected by default. Same input -> identical result asserted (determinism).
- **ToolDefinition**: bad id; unknown category; unknown permission; bad `autonomyFloor`; negative flat cost; empty estimator id; empty wraps; `EXECUTION` tool with `autonomyFloor` $<$ 3; `EXECUTION` tool shipped active.
- **Run transitions**: `queued` -> `succeeded` rejected (must plan first); `succeeded` -> `running` rejected (terminal); `running` -> `queued` rejected; unknown status rejected; every terminal status has an empty outgoing set; every transition target is a known status (table integrity).
- **Evidence**: relevance/confidence outside [0, 1] rejected; missing provenance rejected; empty `provenance.producer` rejected; missing `runId` / `stepId` rejected (traceability).
- **Memory**: unknown layer; retention/policy for a layer not in `layers`; `RUN_STATE` writable through the memory layer - all rejected.
- **Permission**: dangerous permission rejected with no context, with only the env flag, with only the allowlist; allowed only when **both** gates pass.
- **Autonomy**: `DEFAULT` = 0, v1 config max = 2, live-execution floor = 3; `canRunAtAutonomy` boundary cases.

5. Vendor-neutrality audit

Two structural tests in the validation script, plus a repo grep:

1. **No vendor tokens** in any contract file - banned substrings (case- insensitive): @google/genai, openai, anthropic, gemini, claude, deepseek, ollama, langchain, langgraph, @openai/agents. **Zero hits.**
2. **Self-contained imports** - every import / export ... from in the contract layer resolves to a relative ./ path. The contract layer imports **nothing but itself** - no @/types/*, no @/lib/*, no @/services/*, no next/*, no npm package.
3. ModelPolicy uses opaque model-id **strings** resolved by lib/ai at runtime; it does not reference the ProviderName union or any provider type.

Result: no vendor-specific type leaks into the contracts (AN1.2 invariant 7).

6. Contract compatibility notes

- **Legacy layer untouched (AN1.2 D3).** types/agent.ts, services/agents/*, data/mock-agents.ts, config/agent.config.ts, components/agents/*, app/dashboard/agents/page.tsx, app/api/private/agents/route.ts - **not modified.** The new AgentType (registry) and the legacy AgentType (types/agent.ts) are separate symbols in separate modules; neither imports the other.
- **New code is isolated** under types/agent-framework/ and services/agent-framework/. Nothing else in the app imports these yet - A1 adds contracts only.
- **Prisma:** unchanged. The AgentRun / AgentStep / AgentToolCall / AgentEvidence / AgentEvaluation / AgentCreditLedgerEntry / AgentVersion models land in **A3** as a generated-not-applied migration (P1 locked). The contract shapes in agent-run-contract.ts are the source of truth those models will mirror.
- **Evidence vocabulary** deliberately parallels types/evidence.ts (EvidenceItem) - AgentEvidence is a typed sibling recording *which tool result an agent cited*, not a replacement. Nothing is wired into the 15D pipeline.
- **Contract version** AF-v1 is the compatibility anchor for A2-A15.
- **common.ts** added beyond the named 8 files (sec 1) - noted for the record; it holds only shared primitives, no domain shapes.

7. Files changed

Added (11):

```
frontend/types/agent-framework/common.ts
frontend/types/agent-framework/autonomy-contract.ts
frontend/types/agent-framework/permission-contract.ts
frontend/types/agent-framework/evidence-contract.ts
frontend/types/agent-framework/memory-contract.ts
frontend/types/agent-framework/tool-contract.ts
frontend/types/agent-framework/agent-run-contract.ts
frontend/types/agent-framework/agent-contract.ts
frontend/types/agent-framework/index.ts
frontend/services/agent-framework/agent-type-registry.ts
frontend/scripts/validate-agent-contracts.ts
```

Modified (1):

```
frontend/package.json    (+ "validate:agent-contracts" script)
```

Not touched: Prisma schema, every existing route/service/component, the legacy agents UI and its data layer, /dashboard/agents.

8. Test command + result

```
npm run validate:agent-contracts
```

Result: 38 passed, 0 failed.

Typecheck: `npx tsc --noEmit -> 0 errors in types/agent-framework/** or scripts/validate-agent-contracts.ts`. The full-repo run reports 77 errors, all in `.next/dev/types/validator.ts` - a stale Next.js-generated route-types artifact (`.next/` is gitignored; regenerated by `next build`). Pre-existing, unrelated to A1.

9. No-runtime-implementation confirmation

A1 delivered **contracts + registry + validation only**. Enforced structurally by the validation script:

- **No runtime behavior in the contract layer** - banned substrings asserted absent in every `types/agent-framework/*.ts`: `prisma`, `PrismaClient`, `fetch`, `XMLHttpRequest`, `axios`, `node-fetch`, `process.env`, `setTimeout`, `setInterval`, `node-cron`, `@/services/`, `next/server`, `fs.readFile`, `require`.
- **Registry is declarative** - asserted to import only `import type` from `@/types/agent-framework/*`, and to contain none of the above.
- **No execution surface** - `ToolDefinition` has **no handler field**; the three role types (`PlannerToolRequest -> AuthorizedToolIntent -> ExecutorInvocation`) are pure data; `PlannerToolRequest` is asserted to carry no handler/authority field. Binding a tool to an execution handler that calls an existing AT24 service is **A2**.
- **No new API routes, no new Prisma models, no supervisor/planner/runtime code**. Those are A2-A5.

Planner != Executor != Registry is encoded in `tool-contract.ts` as three separate types with a documented one-way flow: `AGENT -> PLANNER (what next) -> PERMISSION POLICY -> TOOL REGISTRY (exists + permitted) -> EXECUTOR (how to run it safely) -> existing AT24 service -> evidence/result`. The planner never holds a handler or invokes application code.

End AN1.3. If clean at G01, proceed to A2 - Tool Contract + Registry.